DATA STRUCTURE

LINKLISTS:-

SIMPLE LINKLIST

```
#include <iostream>
using namespace std;

class Node
{
public:
   int data;
   Node *next;
};

class LinkedList
{
   Node *head;

public:
   LinkedList()
   {
```

```
    head = NULL;
}

void insert(int x)
{
    Node *newnode = new Node;
    newnode->data = x;
    newnode->next = NULL;

    if (head == NULL)
        head = newnode;
    else
    {
        Node *temp = head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
    }
}
```

```
    void display()
    {
        Node *temp = head;

        while (temp != NULL)
        {
            cout << temp->data << " -> ";
            temp = temp->next;
        }

        cout << "NULL";
    }
};

int main()
{
    LinkedList l;

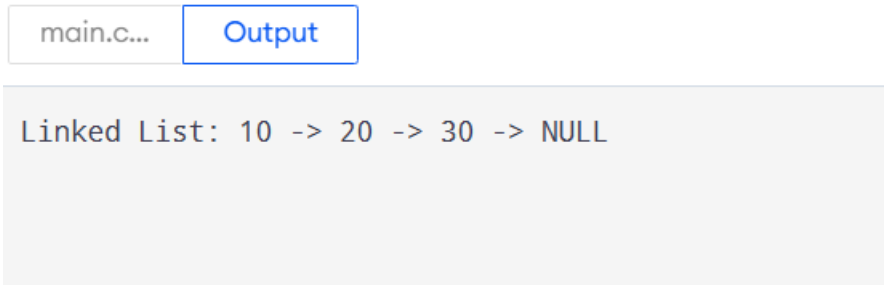
    l.insert(10);
    l.insert(20);
```

```
    l.insert(30);

    cout << "Linked List: ";
    l.display();

    return 0;
}
```

Output:



CIRCULAR LINKLIST

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
```

```
};

class CircularList
{
   Node *head;

public:
   CircularList()
   {
      head = NULL;
   }

   void create(int x)
   {
      Node *newnode = new Node;
      newnode->data = x;

      if (head == NULL)
      {
         head = newnode;
         newnode->next = head;
```

```
    }
    else
    {
        Node *temp = head;

        while (temp->next != head)
            temp = temp->next;

        temp->next = newnode;
        newnode->next = head;
    }
}

void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is Empty";
        return;
    }
```

```
    Node *temp = head;

    if (head->next == head)
    {
        head = NULL;
        delete temp;
    }
    else
    {
        Node *last = head;

        while (last->next != head)
            last = last->next;

        head = head->next;
        last->next = head;

        delete temp;
    }
}
```

```
    void display()
    {
        if (head == NULL)
        {
            cout << "List is Empty";
            return;
        }

        Node *temp = head;

        do
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
        while (temp != head);
    }
};

int main()
{
```

```
    CircularList c;

    c.create(10);
    c.create(20);
    c.create(30);

    cout << "Circular Linked List: ";
    c.display();

    cout << "\nAfter Deletion: ";
    c.deleteNode();
    c.display();

    return 0;
}
```

Output:

```
Circular Linked List: 10 20 30
After Deletion: 20 30
```

LINKLIST WITH STACK

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

class Stack
{
   Node *top;

public:
   Stack()
   {
      top = NULL;
   }
```

```
void push(int x)
{
   Node *newnode = new Node;
   newnode->data = x;
   newnode->next = top;
   top = newnode;
}

void pop()
{
   if (top == NULL)
      cout << "Stack Underflow";
   else
   {
      Node *temp = top;
      cout << "Deleted Element: " << top->data;
      top = top->next;
      delete temp;
   }
}
```

```
    void display()
    {
        Node *temp = top;

        if (top == NULL)
            cout << "Stack is Empty";

        while (temp != NULL)
        {
            cout << temp->data << endl;
            temp = temp->next;
        }
    }
};

int main()
{
    Stack s;

    s.push(10);
    s.push(20);
```

```
    s.push(30);

    cout << "Stack Elements:" << endl;
    s.display();

    cout << "After Pop:" << endl;
    s.pop();

    cout << endl;
    s.display();

    return 0;
}
```

Output:

